# MAS Platform Guide

Human-oriented companion to the authoritative platform specification

MARCH 26, 2026

# Contents

# MAS Platform Guide

This guide is the human-oriented companion to platform-spec.md. It covers the same platform surface, but organizes the material around how a reader usually learns and uses the platform rather than around the raw contract layout.

The authoritative source of truth remains platform-spec.yaml. The prose spec in platform-spec.md is the exact rendering of that YAML. This guide is explanatory. If this guide and the spec ever disagree, the YAML and spec win.

## How To Read This Guide

Read the guide in this order:

1. Start with the platform mental model.
2. Read the contract package model and the flow model together.
3. Read handler execution and engine execution as one runtime story.
4. Read permissions, tools, prompts, compliance, and workspaces as the control surface around that runtime.
5. Use the platform tables and observation model as the "what can I inspect?" layer.

The guide keeps exact identifiers, enum values, and technical terms from the spec. Where the spec is dense, the guide explains how adjacent sections fit together.

## 1. Platform Identity And Vocabulary

The platform is `mas-orchestrator` at version `1.3.0`. The platform vocabulary defines the basic terms used throughout the contracts and runtime.

The key terms are these:

- `state_change` is a transition in the entity state machine, triggered by an `advances_to` handler field.
- `guard` is a boolean check evaluated before handler execution. Guards use CEL expressions evaluated against `entity`, `payload`, and `policy` context. If a guard fails, the handler is blocked.
- `action` is a platform-provided operation executed as the final step of a handler. Only two valid actions exist: `create_flow_instance` and `record_evidence`.
- `timer` is a time-based trigger attached to a stage. When its delay elapses, the platform emits the timer's event, which can trigger a transition.

- `agent_role` is an entity that owns transitions, handles events, and executes logic.
- `flow` is the platform's reusable building block.
- `state` is a named state in a workflow.

### GUARD FAILURE BEHAVIOR

When a guard fails, the platform uses one of these `on_fail_actions`:

- `reject (default)`
- `discard`
- `kill`
- `escalate:{event}`

### AGENT EXECUTION TYPES

The vocabulary distinguishes three execution types:

- `system_node`: deterministic code, no LLM, fixed logic, event-driven orchestration, workflow transition owner.
- `agent`: LLM-powered, prompt-driven, model-backed, event-driven, tool-calling, and able to own transitions where judgment is required.
- `runtime`: the platform itself, handling implicit transitions such as human decisions and lifecycle behavior, without contract declaration.

### TIMER FIELDS

The vocabulary-level `timer` shape defines these required fields:

- `id`
- `state`
- `event`
- `owner`

It also defines these optional fields:

- `delay_seconds`
- `delay_minutes`
- `delay_hours`
- `delay_days`
- `cancellation`
- `recurring`

### FLOW AND STATE BASICS

A `flow` is a self-contained unit with input and output pins, system nodes that orchestrate transitions, and required agent roles. The required files named in the vocabulary are:

- `nodes.yaml`
- `events.yaml`
- `schema.yaml`

A `state` always belongs to a workflow. An entity is always in exactly one state, states belong to phases for grouping, and some states are terminal. The required state fields are:

- `id`
- `phase`

The optional state field is:

- `description`

## 2. The Core Mental Model

The easiest way to understand the MAS platform is to think in layers:

- A `flow` is the unit of composition.
- `schema.yaml` defines the flow contract.
- `nodes.yaml` defines deterministic orchestration.
- `agents.yaml` defines LLM workers and coordinators.
- `events.yaml` defines payload shapes.
- `tools.yaml`, `policy.yaml`, and `prompts/` define the operating environment for agents.
- The engine loads all flows, resolves their paths, validates them, starts subscribers, and runs the event loop.

In practice, the runtime story is:

1. A flow declares states, pins, and required roles.
2. System nodes subscribe to events and own deterministic transitions.
3. Agents subscribe to events and handle reasoning work.
4. Events move work through the system.
5. Handler execution commits atomically.
6. The platform persists state, events, deliveries, receipts, sessions, timers, and other runtime infrastructure in platform-owned tables.

That is the thread connecting the rest of the spec.

## 3. Flow Packages And Contract Files

Every flow uses the same contract layout. `package.yaml` is the manifest. `schema.yaml` is always required. The remaining files are present when the flow needs them.

The canonical layout is:

```
{root_flow}/                        # Root flow (entry point)
  package.yaml                      # Manifest: name, version, flows, handoffs, entity_schema
  schema.yaml                       # Root flow pins
  nodes.yaml                        # Root-level system nodes
  events.yaml                       # Root-level events
  agents.yaml                       # Root-level agents
  tools.yaml                        # Shared tools
  policy.yaml                       # Shared policy
  prompts/                          # Root-level agent prompts
  runtime/                          # Merged flat files for flat file loader
    nodes.yaml
    events.yaml
    agents.yaml
    tools.yaml
    policy.yaml
  flows/
    {child_flow}/                   # Child flow (same structure, recursive)
      package.yaml
      schema.yaml
      nodes.yaml
      events.yaml
      agents.yaml
      tools.yaml                    # Optional
      policy.yaml                   # Optional
      prompts/
      flows/                        # Grandchild flows (if any)
        {grandchild}/
          package.yaml
          ...

platform/                           # Platform contracts (separate from any flow)
  package.yaml
  platform-spec.yaml
```

### MINIMUM PACKAGE RULES

The minimum flow package requires:

- `package.yaml`
- `schema.yaml`

The optional files are:

- `nodes.yaml`
- `events.yaml`
- `agents.yaml`
- `tools.yaml`
- `policy.yaml`

The optional directories are:

- `prompts/`
- `flows/`

The guide-level summary is simple:

- no agents means no `prompts/`
- no handlers means no `nodes.yaml`
- no events means no `events.yaml`

### LOADER DEFAULTS

Several fields are optional because the loader derives them:

- `schema_name`
- `schema_namespace`
- `agent_id`
- `agent_emit_events`

The important identity rule is that the YAML map key in `agents.yaml` is the canonical agent identity when `agent_id` is omitted.

### ENTITY SCHEMA AND EVENT SCHEMA

`entity_schema` in `package.yaml` declares persistent entity fields. The platform derives database DDL from it at boot. The schema is organized into named groups, and its supported types are:

- `text`

- `integer`
- `numeric(precision,scale)`
- `boolean`
- `jsonb`
- `timestamp`
- `uuid`

`events.yaml` defines payload schemas. It does not define routing. Routing is derived from:

- `agents.yaml` subscriptions and `emit_events`
- `nodes.yaml` `subscribes_to` and `event_handlers`

The routing consequences are:

- if a system node subscribes to an event, that node owns it
- if a node and agents subscribe, both receive it
- if only a node subscribes, it is node-only delivery
- if no node subscribes, it is pure agent delivery

### PERSISTENCE TOOLS DERIVED FROM ENTITY SCHEMA

Agents do not get raw database access. Instead, the platform auto-generates typed persistence tools from `entity_schema`. The guide-level list is:

- `get_entity`
- `save_entity_field`
- `search_entities`
- `query_metrics`

### REQUIRED AGENTS

`required_agents` in `schema.yaml` names the roles a flow requires. The fulfillment rule is strict: the role name in `required_agents` must match the top-level agent key in `agents.yaml`.

The spec also allows “empty” required agent declarations. A role may exist with empty subscriptions and emits if it coordinates through universal tools like `agent_message` and `mailbox_send`.

### UNDERSCORE-PREFIXED CONTRACT FIELDS

The contracts use two categories of `_`-prefixed fields:

- semantic fields the platform and verifier understand

- documentation-only fields the platform ignores

The semantic fields are:

- `_source`
- `_consumer`
- `_status`
- `_producer`

Any other `_`-prefixed field is treated as documentation.

## 4. The Flow Model

The spec says a `Flow` is the universal building block. There is no separate project concept. Every level of the system is a flow, from the root flow down to the smallest sub-flow.

### WHAT A FLOW DECLARES

A flow package is defined by:

- `package.yaml`
- `schema.yaml`
- `agents.yaml` when the flow has agents
- `events.yaml` when the flow declares event schemas
- `nodes.yaml` when the flow has system nodes
- `tools.yaml` when the flow needs flow-specific tools
- `policy.yaml` when the flow needs flow-specific configuration
- `prompts/` when the flow has agent prompts
- `flows/` when the flow has child flows

### NESTING

Flows nest recursively. A flow's `package.yaml` names child flows in `flows:`. Each child lives under `flows/` and has the same structure. The root flow is not structurally special. It is the entry point, but it is still just a flow with nodes, agents, events, policy, and child flows.

### PINS

Pins are the public interface of a flow.

The guide version of the pin model is:

- input event pins are the events the flow subscribes to
- output event pins are the events the flow emits
- input data pins are the entity fields the flow reads
- output data pins are the entity fields the flow writes

The spec-level consequences matter:

- required input event pins must be wired
- required input data pins must be available from earlier outputs or initial data
- no two flows may write the same output data pin
- internal events inside a flow are not part of the public pin interface

### REQUIRED AGENTS IN THE FLOW MODEL

At the flow-model level, a `required_agents` entry contains:

- `role`
- `subscribes_to`
- `emits`
- `description`

The platform validates that actual agents fulfill those event contract requirements after namespace resolution.

### CROSS-FLOW EVENT OWNERSHIP

Event schemas are defined once, in the flow that emits them. Consuming flows reference them by absolute path. They do not redefine them. If the same event name appears in multiple flow event files, the emitter flow's definition is authoritative and boot logs a warning.

### STATELESS FLOWS

The spec explicitly allows stateless flows. In a stateless flow:

- `initial_state: null`
- `states: []`
- no `entity_state` rows are created
- `advances_to` is invalid
- guards may reference `payload` and `policy`, but not `entity`
- accumulators are flow-scoped rather than entity-scoped

### STATE COMPOSITION ACROSS FLOWS

Each flow has its own state machine. There is no single global enum for an entity across the entire platform. When an entity moves from one flow to another, one flow emits a cross-flow event and the next flow handles that event into its own state space.

## 5. Hello World Example

The spec's hello world example is the smallest useful illustration of the contract model:

```
name: hello
version: 1.0.0
platform_version: ">=1.1.0"
flows: []
```

```
initial_state: waiting
terminal_states: [done]
states: [waiting, done]
pins:
  inputs:
    events: [hello.requested]
  outputs:
    events: [hello.completed]
```

```
hello-node:
  id: hello-node
  execution_type: system_node
  subscribes_to: [hello.requested]
  produces: [hello.completed]
  event_handlers:
    hello.requested:
      advances_to: done
      emits: hello.completed
```

```
hello.requested:
  payload:
    entity_id: string
    message: string
hello.completed:
  payload:
    entity_id: string
```

The example is useful because it shows the core platform loop in one handler:

- receive an event
- advance state
- emit a follow-up event

## 6. Handler Execution

The handler specification is one of the most important parts of the platform. The spec does not define a 10-step sequential recipe. It defines a dependency graph.

### THE DEPENDENCY GRAPH

This graph is the central model:

```
query (optional — pre-fetch cross-entity data into handler context)
  └→ clear_gates (optional — reset gates before guard)
       └→ guard
            └→ accumulate
                 └→ filter (optional — prune accumulated items)
                      └→ reduce (optional — aggregate filtered items to single value)
                           └→ count (optional — count items)
                                └→ compute
                                     └→ on_complete / rules (mutually exclusive)
                                          └→ advances_to ─┐
                                             sets_gate ────┤ (independent of each other)
                                             data_accumulation ┘
                                                  └→ payload_transform
                                                       └→ emits
                                                            └→ action
                                                                 └→ clear (optional — reset
accumulator/state buckets)
```

The key interpretive point is that arrows mean “must complete before.” Steps at the same level are independent. That is why the spec says YAML field order is cosmetic.

### ATOMICITY

A handler execution is atomic. State change, gate updates, data writes, and event persistence commit in one transaction. Guards read pre-handler state. The current handler’s `data_accumulation` does not affect the current handler’s guard; it affects later handler executions after commit.

### SHORT-CIRCUITS

Handlers can stop early in two important ways:

- `guard_fail`: the handler stops and runs the configured `on_fail` behavior
- `accumulate_incomplete`: the handler records the arrival and stops until completion criteria are met

### ON_COMPLETE AND RULES

These are mutually exclusive branch mechanisms:

- `on_complete` is an ordered list, first match wins, and is used after accumulation or computation
- `rules` is a map used for payload-based routing

### CORE HANDLER FIELDS

The spec calls these the core fields:

- `guard`
- `advances_to`
- `sets_gate`
- `data_accumulation`
- `emits`
- `rules`
- `on_complete`

The extension fields are:

- `accumulate`
- `compute`
- `fan_out`
- `filter`
- `reduce`
- `count`
- `query`
- `clear`
- `payload_transform`
- `clear_gates`
- `action`

### HANDLER FIELDS AT A GLANCE

This is the practical reading of the handler field surface:

- `description`: documentation only
- `guard`: gatekeeper for execution
- `accumulate`: wait for multiple arrivals
- `compute`: run a platform computation primitive
- `on_complete`: post-accumulation branching
- `advances_to`: set `entity.state`
- `sets_gate`: set `entity.gates.{name} = true`
- `data_accumulation`: write payload-derived values into entity state
- `emits`: publish follow-up events
- `rules`: payload-based routing
- `fan_out`: emit per item in a list
- `query`: pre-fetch cross-entity data
- `reduce`: aggregate items
- `filter`: prune items by condition
- `count`: count items
- `clear`: clear state buckets
- `action`: invoke a platform action
- `payload_transform`: construct emitted payloads explicitly
- `clear_gates`: reset all entity gates to `false`
- `evidence_target`: required when `action` is `record_evidence`

### THE ONLY VALID ACTIONS

The platform has exactly two valid handler actions:

- `create_flow_instance`
- `record_evidence`

`create_flow_instance` requires:

- `template`
- `instance_id_from`
- `config_from`

`record_evidence` appends payload data to the target accumulator and requires:

- `evidence_target`

### CANONICAL DATA_ACCUMULATION

The spec gives a canonical shape for `data_accumulation`:

```
data_accumulation:
  writes:
    - field_name
    - source_field: payload_key
      target_field: entity_key
  source_event: event_name
```

It also allows computed writes:

```
data_accumulation:
  writes:
    - field_name
    - source_field: payload_key
      target_field: entity_key
    - target_field: entity_key
      expression: "entity.counter + 1"
  source_event: event_name
```

### RULES AND HANDLER-LEVEL DEFAULTS

When a handler has both `rules` and handler-level fields:

- rule-level side effects execute first
- handler-level `data_accumulation` supplements rule-level writes
- handler-level `emits` supplements rule-level emits
- rule values override handler-level defaults for the same field

## 7. The Engine Runtime

The engine defines how flows are discovered, loaded, validated, started, and executed.

### FLOW TREE WALKER

The flow tree walker starts at the root directory, reads `package.yaml`, registers the flow, descends through `flows:`, and produces a flat list of flow entries. Important constraints include:

- `package.yaml` and `schema.yaml` are always required
- a flow must have either `agents.yaml` or at least one child flow
- maximum nesting depth is `99`
- duplicate flow IDs anywhere in the tree are a boot error

### URI ADDRESSING

The engine gives every addressable runtime entity a path.

Two forms matter day to day:

- local: `{name}`
- absolute: `{flow_id}/{flow_id}/.../{name}`

The rule is simple:

- no slash means local to the current flow
- slash means absolute from the root

The spec also defines wildcard patterns:

- `*/entity.shortlisted`
- `**/entity.completed`

### CONTRACT MERGER

After flow discovery, the engine builds runtime registries. Every node, agent, and event receives a full URI. Tools are flat and shared. Policy is hierarchical, with child overrides of parent values.

### DYNAMIC INSTANCES

Dynamic flow instances are created by handler action, not by ad hoc runtime code. The parent flow must declare the child flow as `mode: template`. At runtime:

1. the handler calls `create_flow_instance`
2. the platform validates the template and `instance_id`
3. the platform loads the template contracts
4. the platform registers nodes, agents, and events
5. the platform creates the entity at the flow's `initial_state`

6. wildcard subscriptions are expanded
7. the new instance starts

The addressing distinction is:

- static flow: `{flow_id}/{local_name}`
- template instance: `{template_id}/{instance_id}/{local_name}`

## BOOT SEQUENCE

The boot sequence moves from `load_platform_spec` to `ready`. The most important checkpoints are:

- walk the flow tree
- construct paths
- register templates
- build registries
- resolve subscriptions
- validate pins
- validate required agents
- validate tools
- validate permissions
- validate platform version
- initialize state stores
- start system nodes
- start agents

If any boot step fails, startup aborts completely.

## EVENT LOOP

The runtime cycle is:

1. an event arrives
2. payload validation runs
3. the event is persisted
4. subscribers are resolved
5. subscribing system nodes handle it
6. subscribing agents receive inbox delivery
7. emitted events are delivered after commit

8. the original event is marked delivered

Important runtime semantics:

- events are serialized per entity
- events across different entities are concurrent
- emitted events from node handlers are persisted inside the atomic boundary but delivered after commit
- agent deliveries are asynchronous

### STATE MANAGEMENT

The engine tracks three state kinds:

- `entity_state`
- `accumulator_state`
- `gate_state`

Terminal states are absorbing. Once an entity reaches a terminal state:

- new events targeting that entity are rejected before handler execution
- timers are cancelled
- agent sessions are terminated
- persisted state remains queryable

### ACCUMULATION ENGINE

The accumulation engine creates and updates accumulator records for handlers using `accumulate`. It tracks received items by `dedup_by`, checks `all`, `threshold`, or `timeout` completion modes, and makes accumulated data available to `on_complete`, `compute`, and later steps.

### AGENT SESSION MANAGEMENT

Agent sessions follow this lifecycle:

1. event arrives in inbox
2. prompt is loaded and variables are substituted
3. tools are attached
4. the agent processes the event
5. tool calls are validated
6. emitted events are validated and published
7. session completes or hits `max_turns_per_task`

8. session state is persisted when applicable

The supported `conversation_mode` values are:

- `task`
- `session`
- `session_per_entity`
- `stateless` as an alias normalized to `task`

## TIMER MODEL

Timers are durable delayed events. They start on a configured state or event, are persisted, fire as regular events, and can be recurring.

The declaration fields are:

- `id`
- `event`
- `delay`
- `recurring`
- `start_on`
- `cancel_on`

## EXPRESSION LANGUAGE

The platform uses CEL for:

- guard checks
- rule conditions
- filter conditions
- `on_complete` branch conditions

The context variables are:

- `entity`
- `payload`
- `policy`
- `fan_out`
- `accumulated`

## ERROR MODEL

The engine distinguishes:

- event validation failure
- handler execution failure
- agent session failure
- timer failure
- chain depth overflow

The chain depth limit is `50`. When it is exceeded, the would-be emitted event is intercepted and moved to `dead_letters`. The triggering handler still succeeds.

## BOOT VERIFICATION

The engine's authoritative boot verification list includes `19` checks. The most important names to recognize are:

- `event_chain_integrity`
- `event_consumer_exists`
- `event_producer_exists`
- `payload_field_coverage`
- `condition_payload_alignment`
- `state_machine_coherence`
- `required_agents_match`
- `handler_field_compliance`
- `tool_resolution`
- `prompt_exists`
- `produces_drift`
- `invalid_field_detection`
- `policy_conflict_detection`
- `event_cycle_detection`
- `dialect_compliance`
- `single_node_per_event`
- `config_from_payload_alignment`
- `phantom_produces`

# 8. Compliance, Permissions, Tools, And Prompts

These four sections define the control layer around execution.

### COMPLIANCE

Compliance defines both boot-time and runtime enforcement.

At runtime, the required checks include:

- tool schema validation before execution
- permission checks before tool execution
- message scope enforcement for agent messaging
- state changes limited to declared `advances_to` paths
- guard execution before state advancement
- payload validation before publish
- idempotent accumulation behavior
- reading permissions from `agent.permissions`
- reading scan modes from `policy.scan_modes`
- reading manager fallback from `agent.manager_fallback`
- reading workspace class from `agent.workspace_class`

The testing section turns those guarantees into required test coverage.

### PERMISSIONS MODEL

Agents have a `permissions` list. The platform enforces permissions at tool execution time and message routing time.

The platform-defined permissions are:

- `agent_fire`
- `agent_hire`
- `agent_reconfigure`
- `approve_spend`
- `configure_routing`
- `create_flow_instance`
- `human_task_decide`
- `human_task_request`
- `mailbox_send`

- `message_flow`
- `message_peers`
- `schedule`

`permissions_bundle` can be combined with explicit `permissions`, and explicit permissions extend the bundle.

### Permission Scoping

The spec is strict here:

- all scoping is flow-instance-local
- no permission grants cross-flow access
- cross-flow communication is via events only

Messaging scope is limited to:

- `message_peers`: agents sharing the same `manager_fallback` value in the same flow instance
- `message_flow`: any agent in the same flow instance

There is no `message_all` and no `message_domain`.

Management scope for `agent_hire`, `agent_fire`, and `agent_reconfigure` follows the `manager_fallback` chain downward within the same flow instance.

Routing scope for `configure_routing` allows self-modification and modification of agents the caller can manage.

## TOOL MODEL

The platform owns tool schema serving. It reads tool definition YAML, generates MCP tool definitions, validates calls, and routes them to handlers.

The platform-builtins list contains:

- `create_flow_instance`
- `record_evidence`
- `create_entity`
- `query_entities`
- `agent_message`
- `mailbox_send`

The subtle but important point is that `create_flow_instance` is a handler action value, not a tool agents can call directly.

The accepted custom tool schema fields are:

- required: `description`
- optional: `handler_type`, `input_schema`, `output_schema`, `parameters`, `returns`

The explicitly not accepted fields are:

- `endpoint`
- `type`

#### PROMPT TEMPLATING

Agent prompts live in `prompts/{agent-id}.md`. Prompt variables use `{{variable_name}}` syntax.

Prompt variables resolve from these sources in priority order:

1. `instance_variables` from `schema.yaml`
2. `policy` values
3. `entity_state` fields
4. runtime tokens

The runtime-token examples named in the spec are:

- `current_date`
- `agent_id`
- `flow_instance_path`

The compliance rule is that prompt variable validation must scan all declared sources.

## 9. Versioning

Platform and products version independently using semver.

For the platform:

- `major` means breaking changes to flow model, handler fields, boot validation, or contract formats
- `minor` means backward-compatible new primitives, fields, or checks
- `patch` means bug fixes and clarifications

For products:

- `major` means breaking flow or event-schema changes
- `minor` means additive agents, events, tools, or threshold changes

- `patch` means prompt improvements, policy tuning, and bug fixes

Products declare `platform_version` in `package.yaml`, and boot enforces compatibility against the running platform version.

## 10. Platform Tables And Runtime Data

The platform tables are the runtime's infrastructure layer. These are not contract-driven entity tables. They are platform-owned tables that exist regardless of which product contracts are loaded.

### THE TABLE SET

The platform tables are:

- `events`
- `event_deliveries`
- `event_receipts`
- `entity_state`
- `agents`
- `agent_sessions`
- `routing_rules`
- `mailbox`
- `spend_ledger`
- `flow_instances`
- `timers`
- `dead_letters`
- `schema_version`

### WHAT EACH TABLE IS FOR

- `events` : append-only event store
- `event_deliveries` : delivery tracking per subscriber
- `event_receipts` : per-handler completion acknowledgements and side effects
- `entity_state` : current entity registry and state store
- `agents` : runtime agent registry
- `agent_sessions` : session state by scope
- `routing_rules` : contract routes and materialized wildcard routes
- `mailbox` : human-in-the-loop task queue

- `spend_ledger` : LLM usage and cost tracking
- `flow_instances` : static and dynamic instance registry
- `timers` : durable timer and scheduled task store
- `dead_letters` : retry exhaustion and chain depth overflow outcomes
- `schema_version` : applied platform schema version

### ADDITIONAL RUNTIME DATA POLICIES

The spec also defines:

- diagnostics encoding inside existing tables
- the rule that `entity_state` is the entity registry
- entity metrics conventions using `entity_state.fields` and `spend_ledger`
- migration policy through `schema_version`
- test bootstrap rules
- credentials policy using `flow_instances.config` or an external vault

For exact DDL, use platform-spec.md. The guide keeps the functional model readable and leaves the full table definitions in the reference spec.

## 11. Observation Model

The observation model defines what a test, audit, or flight recorder can see after handler execution.

### EMITTED EVENTS

The `emitted_events` view includes:

- events from handler `emits`
- events from handler `rules`
- events from handler `on_complete`
- events from handler `fan_out`
- events from handler action side effects such as `auto_emit_on_create`

It excludes:

- agent fixture emissions
- dead-lettered events
- child flow `auto_emit` events from the parent emission log

### PAYLOAD CONSTRUCTION

When `payload_transform` is absent, emitted payloads are constructed from:

1. entity fields
2. trigger payload
3. forced platform fields

The collision priority is:

- platform fields
- `payload_transform`
- trigger payload
- entity fields

### ENTITY STATE OBSERVATION

The observable entity fields after commit are:

- `current_state`
- `gates`
- `fields`
- `accumulator`
- `revision`

### HANDLER OUTCOME

The observable `handler_outcome` values are:

- `success`
- `reject`
- `discard`
- `kill`
- `escalate`
- `dead_letter`
- `terminal_reject`

The final distinction is that system node emissions appear in the triggering handler's observation chain, while agent emissions are independent events entering the loop separately.

# 12. Workspace Model

The workspace model defines the filesystem contract for agent sessions.

### STANDARD MOUNTS

There are exactly three standard mounts:

- `/workspace`
- `/data`
- `/opt/mas/contracts`

`/workspace` is writable and can be either:

- `per-agent`
- `per-flow-instance`

`/data` is always:

- global
- read-only

`/opt/mas/contracts` is always:

- global
- read-only

Products cannot add new mount points. They only define workspace classes that choose the `workspace_scope` for `/workspace`.

### WORKSPACE CLASS DEFINITION

Workspace classes live in `policy.yaml` under `workspace_classes`. Each class defines:

- `description`
- `workspace_scope`

Every `workspace_class` referenced by an agent must exist in policy. Missing definitions are boot errors.

### MOUNT GUARANTEES

The runtime guarantees:

- every agent session has all three standard mounts
- `/data` is identical across all workspace classes

- `/workspace` visibility follows the declared scope
- read-only mounts are enforced at the filesystem level
- mount availability does not vary by `conversation_mode`, `model_tier`, or any other agent property

#### DEPLOYMENT MAPPING

The deployment section explains how the same mount contract maps to:

- local development
- Docker
- production

The portability rule is that contracts should not change across those environments. Only the backing storage implementation changes.

## 13. Crosswalk To The Reference Spec

This guide covers every top-level section from the reference spec:

1. `platform`
2. `vocabulary`
3. `contract_formats`
4. `handler_specification`
5. `engine`
6. `compliance`
7. `permissions_model`
8. `tool_model`
9. `prompt_templating`
10. `flow_model`
11. `versioning`
12. `platform_tables`
13. `observation_model`
14. `workspace_model`

The guide reorganizes them into a learning order:

- identity and vocabulary
- package and contract model

- flow model
- handler execution
- engine runtime
- compliance, permissions, tools, prompts
- versioning
- platform tables and runtime data
- observation
- workspaces

## 14. Recommended Documentation Split

What I recommend is this:

- keep platform-spec.yaml as the sole authority
- keep platform-spec.md as the exact prose rendering of that authority
- use this guide as the reader-first companion for humans

That gives you three layers with clear jobs:

- YAML for exactness
- spec markdown for exact prose reference
- guide markdown for comprehension

The next sections add the practical appendices that make the guide easier to use day to day without weakening the authority boundary.

## 15. Appendix A: Runtime Story

This appendix combines the flow model, boot sequence, event loop, and handler dependency graph into one runtime picture.

#### END-TO-END RUNTIME STORY

```
contracts on disk
  └ package.yaml, schema.yaml, nodes.yaml, events.yaml, agents.yaml, tools.yaml, policy.yaml,
prompts/
       └ flow tree walker discovers every flow
            └ contract merger assigns runtime paths and registries
                 └ boot validation checks pins, handlers, tools, permissions, states, prompts, and
```

```
cycles
                      └─ state stores and platform tables initialize
                           └─ system nodes subscribe
                                └─ agents subscribe
                                     └─ platform ready
                                          └─ event arrives
                                               └─ payload validation
                                                    └─ event persisted
                                                         └─ subscribers resolved
                                                              ├─ node handling
                                                              │    └─ query
                                                              │         └─ clear_gates
                                                              │              └─ guard
                                                              │                   └─ accumulate
                                                              │                        └─ filter
                                                              │                             └─
reduce
                                                              │                                  └─
count

│                                     └─ compute

│                                          └─ on_complete / rules

│                                               └─ advances_to + sets_gate + data_accumulation

│                                                    └─ payload_transform

│                                                         └─ emits

│                                                              └─ action

│                                                                   └─ clear
                                                              └─ agent delivery
                                                                   └─ inbox processing, tool calls,
event emission
```

## WHAT THIS RUNTIME STORY IS SAYING

The story has three distinct phases:

1. Contract loading and boot.
2. Event routing.
3. Subscriber execution.

The boot phase is where the platform establishes:

- flow discovery
- runtime addressing
- subscriptions
- tool registry
- policy inheritance
- validation
- state-store initialization

The event phase is where the platform guarantees:

- every event is validated before publish
- every event is persisted before processing
- system node handling is serialized per entity
- agent delivery is asynchronous

The handler phase is where the platform guarantees:

- dependency-graph execution rather than source-order execution
- atomic commit of handler side effects
- post-commit delivery of emitted events

### THE MINIMAL MENTAL SHORTCUT

If someone needs the shortest faithful summary of the runtime, it is this:

- boot builds a validated flow graph
- events drive all work
- system nodes own deterministic transition logic
- agents own reasoning work
- handlers commit atomically
- emitted events continue the chain after commit

## 16. Appendix B: Contract Author Checklist

This checklist is derived from the boot verification list, flow coherence rules, node coherence rules, agent coherence rules, and runtime compliance expectations.

### PACKAGE-LEVEL CHECKLIST

- `package.yaml` exists.
- `schema.yaml` exists.
- The flow has either `agents.yaml` or child flows.
- Every flow declared in `package.yaml` has a matching directory under `flows/`.
- `platform_version` is compatible with the running platform version.

### SCHEMA-LEVEL CHECKLIST

- `states`, `initial_state`, and `terminal_states` form a coherent state space.
- Input pins are wired by some producer.
- Output data pins do not conflict with another flow's writes.
- `required_agents` roles are fulfilled by actual agent entries.

### EVENT CHECKLIST

- Every emitted event has a payload schema in `events.yaml`.
- Every event in `event_handlers` appears in `subscribes_to`.
- Every event in `emit_events` has a payload schema.
- Every subscription resolves to an event some node or agent produces.
- Cross-flow event consumers reference emitter-owned schemas rather than redefining them.

### NODE CHECKLIST

- Each event is handled by at most one system node.
- `advances_to` values stay inside the flow's state space.
- Guards reference real entity or policy fields.
- `produces`, if present, matches actual emitted events.
- Handler fields use the platform-defined names and shapes only.

### HANDLER CHECKLIST

- `on_complete` and `rules` are not used together in the same handler.
- `data_accumulation.source_event` uses the canonical name `source_event`, not `source`.
- `clear_gates` is understood as all-or-nothing entity gate reset.
- `action` is only `create_flow_instance` or `record_evidence`.
- `evidence_target` is present when `action` is `record_evidence`.

- `template`, `instance_id_from`, and `config_from` are present when `action` is `create_flow_instance`.

### AGENT CHECKLIST

- Every tool in `tools_tier2` exists in the tool registry.
- Agent permissions cover the tools they use.
- Messaging expectations match `message_flow` and `message_peers` only.
- `manager_fallback` is treated as escalation and hierarchy metadata, not as cross-flow messaging permission.
- Every referenced `workspace_class` exists in `policy.yaml`.

### PROMPT AND POLICY CHECKLIST

- Prompt files exist for the agents that require them.
- Prompt variables resolve from allowed sources only.
- Policy keys referenced by guards or prompts exist in the merged policy tree.

### RUNTIME BEHAVIOR CHECKLIST

- Tool calls are schema-validated before execution.
- Tool calls are permission-checked before execution.
- Events are payload-validated before publish.
- Accumulation behavior is idempotent.
- State changes only occur along declared paths.
- Workspace provisioning satisfies `/workspace`, `/data`, and `/opt/mas/contracts`.

## 17. Appendix C: What Changes Where?

This matrix summarizes which contract file is responsible for which kind of declaration.

| File | Primary responsibility | Typical contents |
|---|---|---|
| `package.yaml` | flow manifest and package identity | `name`, `version`, `platform_version`, `flows`, `entity_schema` |
| `schema.yaml` | flow contract surface | states, pins, `required_agents`, `initial_state`, `terminal_states` |
| `nodes.yaml` | deterministic orchestration | `system_node` declarations, `subscribes_to`, `event_handlers`, `timers`, `state_schema`, `permissions` |

| File | Primary responsibility | Typical contents |
|---|---|---|
| `events.yaml` | event payload schemas | event names and payload field types |
| `agents.yaml` | agent registry | subscriptions, `emit_events`, tools, permissions, model and conversation settings |
| `tools.yaml` | tool schema definitions | custom tool descriptions, input and output schemas, handler type |
| `policy.yaml` | flow-scoped configuration | policy keys, bundles, `workspace_classes` |
| `prompts/` | agent prompt bodies | `prompts/{agent-id}.md` files with `{{variable}}` placeholders |

### PRACTICAL READING ORDER FOR AUTHORS

When authoring a new flow, the least confusing sequence is:

1. define the flow lifecycle in `schema.yaml`
2. define event payload shapes in `events.yaml`
3. define deterministic transitions in `nodes.yaml`
4. define required and concrete agents in `schema.yaml` and `agents.yaml`
5. define tool schemas in `tools.yaml`
6. define thresholds, bundles, and workspace classes in `policy.yaml`
7. write prompts in `prompts/`

### THE COMMON MISPLACEMENTS

The spec repeatedly draws these boundaries:

- event routing is not declared in `events.yaml`
- tool endpoint configuration is not part of `tools.yaml`
- cross-flow messaging is not granted by permissions
- `manager_fallback` is not a direct messaging grant
- `create_flow_instance` is not an agent-callable tool

## 18. Appendix D: State And Storage Cheat Sheet

This appendix groups the runtime state and the platform tables by the questions operators and developers usually ask.

### "WHAT STATE DOES AN ENTITY HAVE?"

The spec defines three persisted state kinds per entity instance:

- `entity_state`
- `accumulator_state`
- `gate_state`

The practical mapping is:

- lifecycle position lives in `entity_state.current_state`
- gate flags live in `entity_state.gates`
- contract-visible entity fields live in `entity_state.fields`
- accumulator material may live in `entity_state.accumulator` and per-node state storage, depending on the handler pattern

### "WHERE DO I LOOK FOR EVENT HISTORY?"

- raw event history is in `events`
- per-subscriber delivery status is in `event_deliveries`
- per-handler outcomes and side effects are in `event_receipts`
- retry-exhausted and chain-depth-overflow failures are in `dead_letters`

### "WHERE DO I LOOK FOR FLOW ROUTING?"

Use `routing_rules`.

It contains both:

- contract-defined routing patterns
- materialized concrete routes for dynamic instances

The lifecycle is:

- boot inserts pattern rows
- `create_flow_instance` inserts materialized rows
- instance termination inactivates materialized rows

### "WHERE DO I LOOK FOR SESSIONS AND AGENTS?"

- current runtime agents are in `agents`
- session state is in `agent_sessions`
- session lookup is by `(agent_id, scope_key)`

The session scopes are:

- entity scope
- flow scope
- global scope

### "WHERE DO I LOOK FOR HUMAN TASKS?"

Use `mailbox`.

That is where the platform stores:

- `human_task`
- `review_request`
- `approval`
- `alert`
- `operational_decision`

### "WHERE DO I LOOK FOR SPEND?"

Use `spend_ledger`.

It stores one row per agent invocation and tracks:

- `input_tokens`
- `output_tokens`
- `cost_usd`
- `model`
- `invocation_type`

### "WHERE DO I LOOK FOR DYNAMIC FLOW INSTANCES?"

Use `flow_instances`.

That is where the platform tracks:

- `instance_id`
- `flow_template`
- `mode`
- `parent_instance`
- `config`
- `status`

### "WHERE DO I LOOK FOR TIMERS?"

Use `timers`.

That table covers:

- entity-scoped timers
- flow-scoped timers
- global timers

The important task types are:

- `timer`
- `scheduled_task`
- `deadline`
- `global_recurring`

### "WHERE DO I LOOK FOR SCHEMA LIFECYCLE?"

Use `schema_version`.

That table records the currently applied platform schema version and the migration log. It is the table the boot sequence consults before applying platform DDL changes.